DocMind — Technical PRD

Oct 27, 2026 · @Rohan

1. Executive Summary & Problem Statement

DocMind is a self-hosted, multimodal RAG platform that answers compliance questions only with claims it can trace to a page, a bounding box and a verbatim span in the source document, and abstains when it cannot. It targets regulated teams (legal, risk, audit, quality) that cannot ship an answer they cannot defend.

Scope note on "zero hallucination". No LLM pipeline can guarantee zero hallucination. DocMind's contract is narrower and testable: *no unsupported claim is shown as supported.* Every sentence is either (a) entailed by cited evidence and verified, (b) explicitly flagged as unverified, or (c) replaced by an abstention. Section 8 defines how this is measured.

1.1 Why naive RAG fails in enterprise compliance

Failure mode	What happens in a naive pipeline	Compliance consequence	DocMind countermeasure
Fixed-size chunking	Splits clauses, tables and footnotes mid-sentence; definitions separated from the clauses that use them	Answer cites the clause but omits its carve-out	Layout-aware, structure-preserving chunking with parent-child context windows
Dense-only retrieval	Misses exact identifiers ("Art. 28(3)(a)", "ISO 27001 A.8.24", "SOC 2 CC6.1") and rare terms	Wrong or missing regulation retrieved	Hybrid dense + BM25 sparse with RRF fusion
Scanned PDFs and tables	Text extraction returns garbage or flattens tables into run-on text	Numbers and thresholds silently wrong	PyMuPDF native text + PaddleOCR fallback + table-structure recovery

Failure mode	What happens in a naive pipeline	Compliance consequence	DocMind countermeasure
No reranking	Top-k by bi-encoder similarity includes near-duplicate, superseded or tangential chunks	Stale version of a policy is cited	Cross-encoder reranker + version/effective-date filters
Unverified generation	LLM paraphrases beyond the evidence, merges two clauses, invents a section number	Fabricated citation in an audit trail	Dual-agent debate: claim extraction, evidence entailment check, hallucination score, abstain
Citations as strings	Model writes "see section 4.2" with no link to the source region	Reviewer cannot verify in under a minute	Citations are structured objects (doc, page, bbox, span) rendered as highlights in the PDF
No audit trail	Query, evidence and answer not retained	Cannot reconstruct why an answer was given	Immutable audit log of every query, retrieved set, verdict and model version
Repeated identical questions	Every query pays full retrieval + LLM cost	Latency and GPU cost scale linearly with users	Semantic cache keyed by embedding, scoped by tenant and corpus version

1.2 Goals

1. Answer compliance questions with claim-level citations that resolve to a highlighted region of the source PDF.
2. Abstain, rather than guess, when evidence is insufficient or contradictory.
3. Run fully on-prem (Docker Compose on an Arch Linux host) with no document content leaving the network.
4. Provide an auditable, exportable record of every answer.
5. Retrieval path (cache miss) p95 under 150 ms; cache-hit path p95 under 25 ms.

1.3 Non-goals (v1)

- Legal advice or automated regulatory determinations; DocMind surfaces evidence, humans decide.
- Training or fine-tuning foundation models.
- Real-time collaborative editing of source documents.
- Multi-region active-active deployment.

1.4 Success metrics

Metric	Target	How measured
Citation precision (cited span supports claim)	⩾ 98%	Weekly human audit of 200 sampled answers + automated NLI check
Unsupported-claim leak rate (unsupported claim shown as verified)	⩽ 0.5% of claims	Gold set of 500 adversarial questions with known-absent answers
Retrieval recall@20 on gold set	⩾ 95%	Offline eval harness, nightly
Abstention correctness (abstains when answer is absent)	⩾ 95%	Same gold set
Retrieval + rerank latency p95	< 150 ms	Prometheus histogram, cache-miss path
End-to-end time to first token p95	< 1.5 s	Client-side timing
Semantic cache hit ratio (steady state)	⩾ 35%	Redis counters per tenant

1.5 Personas

- **Compliance analyst:** asks questions, needs to verify every claim in one click.
- **Auditor:** replays historical answers, exports evidence packs.
- **Knowledge admin:** ingests and versions documents, monitors parsing quality.
- **Platform engineer:** operates the stack, watches latency, cache and GPU budgets.

2. System Architecture & Data Flow

DocMind splits into an asynchronous **ingestion plane** (GPU batch work, minutes per document) and a latency-critical **query plane** (retrieval in under 150 ms, verification after). The two share Qdrant, Redis and PostgreSQL, and nothing else.

2.1 End-to-end diagram (Mermaid)

```
flowchart TB
  subgraph CLIENT["Next.js 15 client"]
    UI["Explorer, Chat, PDF viewer, Graph view"]
  end

  subgraph GATEWAY["FastAPI gateway"]
    GW["AuthN/Z, tenant scoping, rate limit, request id"]
  end

  subgraph INGEST["Ingestion plane - async workers"]
    UP["POST /api/ingest"] --> JQ["Redis Streams: ingest.jobs"]
    JQ --> W1["Ingest worker"]
    W1 --> DEDUP["SHA-256 dedupe and version check"]
    DEDUP --> KIND{"Text layer per page?"}
    KIND -->|"yes"| FITZ["PyMuPDF: text, spans, bboxes"]
    KIND -->|"no or low density"| OCR["PaddleOCR: detect, recognize, table
structure"]
    FITZ --> LAYOUT["Layout normalizer: headings, paragraphs, tables,
figures"]
    OCR --> LAYOUT
    LAYOUT --> CHUNK["Structure-aware chunker: child 256 tok, parent 1024
tok"]
    CHUNK --> ENRICH["Enrichment: section path, defined terms, effective
date, doc version"]
    ENRICH --> DENSEEMB["BGE-M3 dense embeddings, 1024-d, GPU batch"]
    ENRICH --> SPARSEEMB["BM25 sparse vectors: tokenize, hash, term
frequency"]
    DENSEEMB --> UPSERT["Qdrant upsert: named vectors dense + bm25"]
    SPARSEEMB --> UPSERT
    ENRICH --> PGW[("PostgreSQL: documents, pages, chunks")]
    UPSERT --> BUMP["Increment corpus_version, invalidate cache scope"]
    UPSERT --> GRAPHJOB["Graph job: UMAP + HDBSCAN + similarity edges"]
    GRAPHJOB --> PGW
  end
```

```
  subgraph QUERY["Query plane"]
    QIN["POST /api/query"] --> QEMB["Embed query with BGE-M3"]
    QEMB --> CACHE{"Semantic cache lookup in Redis"}
    CACHE -->|"hit: cosine >= 0.95, same tenant, same corpus_version"|
CACHED["Return cached verified answer"]
    CACHE -->|"miss"| REWRITE["Query rewrite and decomposition, optional"]
    REWRITE --> DSEARCH["Qdrant dense ANN, top 50"]
    REWRITE --> SSEARCH["Qdrant sparse BM25, top 50"]
    DSEARCH --> RRF["Reciprocal Rank Fusion, k=60, top 40"]
    SSEARCH --> RRF
    RRF --> RERANK["Cross-encoder reranker bge-reranker-v2-m3, top 8"]
    RERANK --> EXPAND["Parent expansion, dedupe, version filter"]
    EXPAND --> SYNTH["Synthesizer: Llama-3.3-70B or DeepSeek-V3, claim-tagged
citations"]
  end

  subgraph DEBATE["Multi-agent fact verification debate"]
    CLAIMS["Claim extractor"] --> AGA["Agent A, Advocate: find supporting
span per claim"]
    CLAIMS --> AGB["Agent B, Skeptic: find contradiction or overreach per
claim"]
    AGA --> JUDGE["Judge: per-claim verdict and hallucination score"]
    AGB --> JUDGE
    JUDGE --> GATE{"Score gate"}
    GATE -->|"pass"| FINAL["Final answer with bbox citations"]
    GATE -->|"repairable, max 1 round"| SYNTH
    GATE -->|"fail"| ABSTAIN["Abstain or partial answer with flags"]
  end

  UI --> GW
  GW --> UP
  GW --> QIN
  SYNTH --> CLAIMS
  FINAL --> CWRITE["Write cache: verified answers only"]
  FINAL --> AUDIT[("PostgreSQL: append-only audit log")]
  ABSTAIN --> AUDIT
  CWRITE --> REDIS[("Redis: cache, streams, rate limits")]
  CACHE -.-> REDIS
  UPSERT -.-> QDRANT[("Qdrant")]
  DSEARCH -.-> QDRANT
```

```
  SSEARCH -.-> QDRANT
  FINAL --> UI
  CACHED --> UI
  ABSTAIN --> UI
```

2.2 Components and responsibilities

Component	Technology	Responsibility	Scales by
API gateway	FastAPI, Uvicorn, Pydantic v2	Auth, validation, SSE streaming, orchestration	Stateless replicas
Ingest worker	Python 3.11, PyMuPDF, PaddleOCR, Unstructured (fallback for DOCX/HTML/PPTX)	Parse, OCR, chunk, embed, index	Worker count per GPU
Dense embedder	BGE-M3 via sentence-transformers or Text Embeddings Inference	1024-d vectors, fp16, batch 64	One GPU service, shared
Sparse encoder	BM25 term-weight vectors, IDF applied by Qdrant	Exact identifiers, clause numbers, rare terms	CPU
Vector store	Qdrant (single node, then 3-node cluster)	HNSW dense + sparse inverted index, payload filters	Shards by tenant or hash
Semantic cache	Redis 8 or Redis Stack with vector index (HNSW)	Embedding-keyed answer cache, TTL, corpus-version scoping	Memory
Job queue	Redis Streams, consumer groups	Ingest jobs, retries, dead-letter	Consumers
Reranker	bge-reranker-v2-m3 (cross-encoder), fp16	Re-score fused top 40 against the query	Shared GPU, micro-batching
LLM serving	vLLM (OpenAI-compatible API)	Synthesis, claim extraction, debate agents, judge	GPU tensor parallel

Component	Technology	Responsibility	Scales by
Metadata DB	PostgreSQL 16 (SQLite for single-user dev)	Documents, pages, chunks, audit, graph	Vertical, read replica
Observability	Prometheus, Grafana, OpenTelemetry, Loki	Latency histograms, cache ratios, GPU use, traces	n/a

2.3 Data flow, step by step

1. **Ingest.** The client uploads a PDF. The gateway stores the file, writes a `documents` row with status `queued`, and enqueues a job. The call returns `202` with a job id.
2. **Parse.** Each page is classified. Pages with a usable text layer go through PyMuPDF, which keeps span-level bounding boxes. Scanned pages go through PaddleOCR, including table structure recovery.
3. **Chunk.** Blocks are merged into child chunks of about 256 tokens that never cross a heading or split a table row. Each child points to a parent of about 1024 tokens. Tables are stored as their own chunk type, in Markdown plus the raw cell grid.
4. **Index.** Each child gets a dense vector and a BM25 sparse vector, upserted as one Qdrant point with named vectors. The `corpus_version` counter for the tenant is incremented, which invalidates cached answers.
5. **Query.** The query is embedded once and reused for the cache lookup and dense search. On a hit the cached, already-verified answer is returned.
6. **Retrieve.** Dense and sparse searches run in parallel. RRF fuses them. The cross-encoder rescores the top 40 and keeps 8.
7. **Synthesize.** The LLM answers using only the supplied evidence and tags each sentence with chunk ids.
8. **Verify.** Claims are extracted and put through a two-agent debate. A judge assigns per-claim verdicts and a hallucination score.
9. **Deliver.** The answer streams to the client immediately, marked *pending verification*. Per-claim verdicts arrive as follow-up SSE events and upgrade or strike each sentence. Only fully verified answers are written to the cache.

2.4 Key architecture decisions

Decision	Choice	Reason	Trade-off
Fusion location	Application-side RRF (weighted)	Tunable weights per query type, debuggable rank traces for the audit log	Two round trips to Qdrant; mitigated by parallel calls. Server-side `Fusion.RRF` is the fallback.
What is cached	Verified answers only	A cached hallucination is served at 25 ms to everyone	Lower hit ratio on contested questions
Cache invalidation	`corpus_version` in the cache key and index filter	Any ingest or deletion makes stale answers unreachable without a scan	Frequent ingest lowers hit ratio; per-collection versions soften this
Chunking	Child/parent, structure-aware	Precision of small chunks at search time, context of large chunks at synthesis time	More storage; two lookups
Verification timing	After streaming starts, with strikethrough on failure	Keeps time to first token under 1.5 s	Users may briefly read an unverified sentence; UI marks it clearly. A strict mode holds the stream until the verdict.
Debate model	Same served LLM, separate system prompts, temperature differs per role	One GPU footprint, shared KV cache for the evidence prefix	Correlated errors; an optional smaller independent NLI model is added as a third check
Metadata store	PostgreSQL in production, SQLite via the same SQLAlchemy models in dev	Audit log needs transactions and append-only permissions	Two dialects to test

2.5 Deployment topology (Docker Compose on Arch Linux)

Services: `api`, `worker`, `vllm`, `tei-embed`, `tei-rerank`, `qdrant`, `redis`, `postgres`, `web`, `prometheus`, `grafana`. GPU services need the `nvidia-container-toolkit` package from the Arch repositories and `deploy.resources.reservations.devices` in Compose. Qdrant and PostgreSQL use named volumes on NVMe. All services join one internal network; only `web` and `api` are published behind a reverse proxy (Caddy or Traefik) with TLS.

3. Database & Vector Schemas

Three stores, one rule: **PostgreSQL is the system of record, Qdrant and Redis are rebuildable.** Any vector or cache entry can be regenerated from PostgreSQL plus the original files.

3.1 Qdrant collection

One collection holds all tenants, partitioned by a tenant-indexed payload field. With `m=0` at collection level and `payload_m=16`, Qdrant builds HNSW sub-graphs per tenant, so filtered search stays fast and tenants do not pollute each other's graph. Every query must carry a `tenant_id` filter; the API layer enforces it.

```
from qdrant_client import QdrantClient, models

client = QdrantClient(url="http://qdrant:6333", prefer_grpc=True)

client.create_collection(
    collection_name="docmind_chunks_v1",
    vectors_config={
        "dense": models.VectorParams(
            size=1024,                       # BGE-M3 dense dimension
            distance=models.Distance.COSINE,
            datatype=models.Datatype.FLOAT16, # halves RAM, negligible recall
loss
            on_disk=False,
        ),
    },
    sparse_vectors_config={
        "bm25": models.SparseVectorParams(
            modifier=models.Modifier.IDF,     # Qdrant applies IDF at query
time
```

```
            index=models.SparseIndexParams(on_disk=False),
        ),
    },
    hnsw_config=models.HnswConfigDiff(
        m=0,               # no global graph: all queries are tenant-scoped
        payload_m=16,      # per-tenant graph connectivity
        ef_construct=200,  # build-time quality
        full_scan_threshold=10_000,  # small tenants use exact scan
    ),
    quantization_config=models.ScalarQuantization(
        scalar=models.ScalarQuantizationConfig(
            type=models.ScalarType.INT8, quantile=0.99, always_ram=True
        )
    ),
    optimizers_config=models.OptimizersConfigDiff(default_segment_number=4),
    on_disk_payload=True,
)

# Payload indexes (tenant index must be created first)
client.create_payload_index("docmind_chunks_v1", "tenant_id",
    models.KeywordIndexParams(type="keyword", is_tenant=True))
for field, schema in [
    ("doc_id", "keyword"), ("chunk_type", "keyword"), ("acl_groups",
"keyword"),
    ("collection_ids", "keyword"), ("superseded", "bool"),
    ("effective_date", "datetime"), ("page_start", "integer"),
]:
    client.create_payload_index("docmind_chunks_v1", field, schema)
```

Search-time parameters: `hnsw_ef=128`, quantization `rescore=True` with `oversampling=2.0`. Raise `hnsw_ef` to 256 for the offline evaluation harness only.

Point layout. The point id is a UUIDv5 of `tenant_id/doc_id/version/chunk_index`, so re-ingesting the same document version is idempotent.

Payload field	Type	Purpose
`tenant_id`	keyword (tenant index)	Isolation boundary, always filtered
`doc_id`, `doc_version`	keyword, int	Join back to PostgreSQL; version pinning

Payload field	Type	Purpose
`chunk_id`, `parent_id`	keyword	Parent expansion at synthesis time
`chunk_type`	keyword	`text`, `table`, `list`, `figure_caption`, `footnote`
`text`	string (on disk)	Child chunk text, returned without a PostgreSQL round trip
`section_path`	keyword[]	e.g. `["Article 28", "28(3)", "(a)"]` for display and filtering
`page_start`, `page_end`	int	Citation page
`bboxes`	object[]	`{page, x0, y0, x1, y1}` normalized 0 to 1, origin top-left
`effective_date`, `superseded`	datetime, bool	Excludes outdated policies by default
`classification`, `acl_groups`	keyword, keyword[]	Document-level access control, filtered at query time
`collection_ids`	keyword[]	User-defined corpora for scoped search
`parse_method`, `ocr_conf`	keyword, float	Lets the reranker and judge discount low-confidence OCR
`token_count`, `content_hash`	int, keyword	Budgeting and dedupe

3.2 Redis schema

Redis 8 (or Redis Stack) with the vector index module. `{tenant}` in key names is a hash tag so a tenant's keys co-locate if the deployment moves to Redis Cluster.

Key pattern	Type	Contents	TTL
`dm:{tenant}:cv`	string (int)	Corpus version; `INCR` on every ingest, delete or supersede	none
`dm:{tenant}:l1:{sha256}`	string (JSON)	Exact-match cache. Hash of normalized question + filters + `corpus_version` + `model_sig`	24 h

Key pattern	Type	Contents	TTL
`dm:sc:{tenant}:{uuid}`	hash	Semantic cache entry (fields below)	7 d
`dm:stream:ingest`	stream	Ingest jobs; consumer group `ingest-workers`	trimmed by length
`dm:stream:ingest:dlq`	stream	Jobs that failed 3 times	30 d
`dm:job:{job_id}`	hash	`status`, `stage`, `progress`, `error`, `doc_id`	7 d
`dm:q:{request_id}:events`	stream	SSE events for resumable query streams	1 h
`dm:rl:{tenant}:{user}:{minute}`	string (int)	Rate limit counter	120 s
`dm:lock:doc:{doc_id}`	string	Ingest lock, `SET NX PX`	15 min

Semantic cache entry (`dm:sc:{tenant}:{uuid}` hash):

Field	Type	Notes
`embedding`	binary FLOAT32[1024]	BGE-M3 query embedding
`tenant`	tag	Mandatory filter
`corpus_version`	numeric	Entry is valid only if equal to the current `cv`
`model_sig`	tag	Hash of LLM name, prompt version, reranker version, chunker version
`filters_hash`	tag	Different collection or date filter = different entry
`question`	text	Original question, for the cache inspector UI
`answer_json`	string	Full `QueryResponse` including citations and verdicts
`halluc_score`	numeric	Only entries with score at or below 0.10 are ever written

Field	Type	Notes
`hits`, `created_at`	numeric	Eviction and analytics

```
FT.CREATE idx:sc ON HASH PREFIX 1 "dm:sc:" SCHEMA
  tenant TAG corpus_version NUMERIC model_sig TAG filters_hash TAG
  embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 1024 DISTANCE_METRIC COSINE

FT.SEARCH idx:sc "(@tenant:{t1} @corpus_version:[42 42] @model_sig:{ab12}
@filters_hash:{f0})=>[KNN 1 @embedding $vec AS dist]"
  PARAMS 2 vec <blob> SORTBY dist DIALECT 2
```

A hit requires `dist <= 0.05` (cosine similarity at or above 0.95). The threshold is configurable per tenant and calibrated on a paraphrase set; see Section 8.

3.3 PostgreSQL metadata schema

The same SQLAlchemy 2.0 models run on SQLite for single-user development (JSONB becomes JSON, arrays become JSON, UUID becomes CHAR(36)).

```
CREATE TABLE documents (
  doc_id          UUID PRIMARY KEY,
  tenant_id       TEXT NOT NULL,
  title           TEXT NOT NULL,
  source_filename TEXT NOT NULL,
  sha256          CHAR(64) NOT NULL,
  mime_type       TEXT NOT NULL,
  storage_key     TEXT NOT NULL,                -- object store or volume
path
  page_count      INT,
  status          TEXT NOT NULL CHECK (status IN

('queued','parsing','ocr','chunking','embedding','indexed','failed','deleted'
)),
  current_version INT  NOT NULL DEFAULT 1,
  classification  TEXT NOT NULL DEFAULT 'internal',
  acl_groups      TEXT[] NOT NULL DEFAULT '{}',
  collection_ids  TEXT[] NOT NULL DEFAULT '{}',
  effective_date  DATE,
  superseded_by   UUID REFERENCES documents(doc_id),
  error           JSONB,
```

```
  created_by      TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, sha256)
);
CREATE INDEX ix_documents_tenant_status ON documents (tenant_id, status);

CREATE TABLE pages (
  doc_id UUID NOT NULL REFERENCES documents ON DELETE CASCADE,
  version INT NOT NULL,
  page_no INT NOT NULL,                              -- 1-based
  width_pt REAL NOT NULL, height_pt REAL NOT NULL, rotation SMALLINT NOT NULL
DEFAULT 0,
  parse_method TEXT NOT NULL CHECK (parse_method IN ('native','ocr')),
  ocr_mean_conf REAL,
  PRIMARY KEY (doc_id, version, page_no)
);

CREATE TABLE chunks (
  chunk_id     UUID PRIMARY KEY,
  doc_id       UUID NOT NULL REFERENCES documents ON DELETE CASCADE,
  version      INT  NOT NULL,
  parent_id    UUID REFERENCES chunks(chunk_id),   -- NULL for parents
  chunk_index  INT  NOT NULL,
  chunk_type   TEXT NOT NULL,
  text         TEXT NOT NULL,
  token_count  INT  NOT NULL,
  page_start   INT  NOT NULL, page_end INT NOT NULL,
  bboxes       JSONB NOT NULL,                       -- [{page,x0,y0,x1,y1}]
  section_path TEXT[] NOT NULL DEFAULT '{}',
  content_hash CHAR(64) NOT NULL,
  qdrant_id    UUID NOT NULL,
  UNIQUE (doc_id, version, chunk_index)
);
CREATE INDEX ix_chunks_doc ON chunks (doc_id, version);

CREATE TABLE queries (
  query_id UUID PRIMARY KEY, tenant_id TEXT NOT NULL, user_id TEXT NOT NULL,
  question TEXT NOT NULL, filters JSONB NOT NULL DEFAULT '{}',
  mode TEXT NOT NULL CHECK (mode IN ('fast','strict')),
  corpus_version BIGINT NOT NULL,
  cache_status TEXT NOT NULL CHECK (cache_status IN
('hit_l1','hit_semantic','miss')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

CREATE TABLE answers (
  answer_id UUID PRIMARY KEY, query_id UUID NOT NULL REFERENCES queries,
  answer_text TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('verified','partial','abstained')),
  hallucination_score REAL NOT NULL,              -- 0 best, 1 worst
  model_sig JSONB NOT NULL,                       -- llm, prompts, reranker,
chunker versions
  retrieval_trace JSONB NOT NULL,                 -- dense ranks, sparse
ranks, rrf, rerank scores
  latency_ms JSONB NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE claims (
  claim_id UUID PRIMARY KEY, answer_id UUID NOT NULL REFERENCES answers,
  ordinal INT NOT NULL, claim_text TEXT NOT NULL,
  verdict TEXT NOT NULL CHECK (verdict IN
          ('supported','partially_supported','unsupported','contradicted')),
  confidence REAL NOT NULL,
  advocate JSONB NOT NULL, skeptic JSONB NOT NULL, judge_rationale TEXT NOT
NULL
);

CREATE TABLE citations (
  citation_id UUID PRIMARY KEY, claim_id UUID NOT NULL REFERENCES claims,
  chunk_id UUID NOT NULL REFERENCES chunks, doc_id UUID NOT NULL, page_no INT
NOT NULL,
  bbox JSONB NOT NULL, quote TEXT NOT NULL,
  quote_start INT NOT NULL, quote_end INT NOT NULL,   -- char offsets inside
chunks.text
  entailment_score REAL NOT NULL
);

CREATE TABLE graph_nodes (
  node_id UUID PRIMARY KEY, tenant_id TEXT NOT NULL, graph_version BIGINT NOT
NULL,
  kind TEXT NOT NULL CHECK (kind IN ('document','chunk','cluster')),
  ref_id UUID NOT NULL, label TEXT NOT NULL,
  x REAL NOT NULL, y REAL NOT NULL, cluster_id INT, size REAL NOT NULL
DEFAULT 1
);
```

```
CREATE TABLE graph_edges (
  edge_id UUID PRIMARY KEY, tenant_id TEXT NOT NULL, graph_version BIGINT NOT
NULL,
  src UUID NOT NULL, dst UUID NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN
('similar','cites','supersedes','references')),
  weight REAL NOT NULL
);

-- Tamper-evident audit log: hash chain, application role has INSERT only
CREATE TABLE audit_log (
  seq BIGSERIAL PRIMARY KEY, ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  tenant_id TEXT NOT NULL, actor TEXT NOT NULL,
  action TEXT NOT NULL,                          -- ingest, delete, query,
verify, export
  entity_type TEXT NOT NULL, entity_id UUID NOT NULL,
  payload JSONB NOT NULL,
  prev_hash CHAR(64) NOT NULL, row_hash CHAR(64) NOT NULL  --
sha256(prev_hash || canonical row)
);
REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM docmind_app;
```

Retention: `queries`, `answers`, `claims` and `citations` are kept for the tenant's configured audit window (default 7 years for regulated tenants); chunk text for deleted documents is purged, while the audit log keeps hashes and ids only.

4. API Specification

All endpoints live under `/api`, require a bearer JWT carrying `tenant_id`, `sub` and `groups`, and return `application/json` unless stated. Models are Pydantic v2. Every response carries an `X-Request-ID` header, which is also written to the audit log.

Method and path	Purpose	Success
`POST /api/ingest`	Upload a document and start parsing	`202` + job
`GET /api/ingest/{job_id}`	Poll ingestion progress	`200`
`POST /api/query`	Ask a question; JSON, or SSE with `Accept: text/event-stream`	`200`

Method and path	Purpose	Success
`GET /api/audit`	Search past answers	`200`
`GET /api/audit/{answer_id}`	Full replay: evidence, claims, debate transcripts	`200`
`POST /api/audit/verify`	Re-verify a stored answer or an arbitrary draft text	`200`
`GET /api/audit/integrity`	Check the audit hash chain	`200`
`GET /api/graph`	Nodes and edges for the cluster view	`200`
`POST /api/graph/rebuild`	Recompute layout and clusters	`202`
`GET /api/documents`, `GET /api/documents/{id}`, `GET /api/documents/{id}/pdf`, `DELETE /api/documents/{id}`	Explorer and PDF viewer support	`200` / `204`

4.1 Shared models

```
from __future__ import annotations
from datetime import date, datetime
from enum import StrEnum
from typing import Literal
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field, model_validator

class Strict(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

class Classification(StrEnum):
    public = "public"; internal = "internal"
    confidential = "confidential"; restricted = "restricted"

class BBox(Strict):
    """Normalized page coordinates, origin top-left, 0..1."""
    page: int = Field(ge=1)
    x0: float = Field(ge=0, le=1); y0: float = Field(ge=0, le=1)
    x1: float = Field(ge=0, le=1); y1: float = Field(ge=0, le=1)
```

```
    @model_validator(mode="after")
    def ordered(self):
        if self.x1 <= self.x0 or self.y1 <= self.y0:
            raise ValueError("bbox must have x1>x0 and y1>y0")
        return self

class Filters(Strict):
    collection_ids: list[str] = Field(default_factory=list, max_length=50)
    doc_ids: list[UUID] = Field(default_factory=list, max_length=200)
    effective_on: date | None = None        # point-in-time: versions in
force on this date
    include_superseded: bool = False
```

4.2 `POST /api/ingest`

Request is `multipart/form-data`: `file` (PDF, max 200 MB; DOCX, PPTX, HTML via Unstructured) and `options` (a JSON string validated against `IngestOptions`). Optional `Idempotency-Key` header makes retries safe.

```
class IngestOptions(Strict):
    title: str | None = Field(None, max_length=300)
    collection_ids: list[str] = Field(default_factory=list, max_length=20)
    classification: Classification = Classification.internal
    acl_groups: list[str] = Field(default_factory=list, max_length=50)
    effective_date: date | None = None
    supersedes_doc_id: UUID | None = None   # marks the older doc superseded
after indexing
    ocr: Literal["auto", "force", "off"] = "auto"
    languages: list[str] = Field(default=["en"], max_length=5)   # PaddleOCR
language packs

class IngestAccepted(Strict):
    job_id: UUID
    doc_id: UUID
    status: Literal["queued"] = "queued"
    duplicate_of: UUID | None = None        # same sha256 already indexed for
this tenant
    status_url: str

class IngestStatus(Strict):
    job_id: UUID
    doc_id: UUID
```

```
    status: Literal["queued", "parsing", "ocr", "chunking", "embedding",
                    "indexed", "failed"]
    progress: float = Field(ge=0, le=1)
    pages_total: int | None = None
    pages_ocr: int | None = None
    chunks_indexed: int | None = None
    warnings: list[str] = Field(default_factory=list)   # e.g. "page 14: OCR
confidence 0.62"
    error: ErrorBody | None = None
    finished_at: datetime | None = None
```

4.3 `POST /api/query`

```
class QueryRequest(Strict):
    question: str = Field(min_length=3, max_length=2000)
    filters: Filters = Field(default_factory=Filters)
    mode: Literal["fast", "strict"] = "fast"   # strict holds output until
verification ends
    context_chunks: int = Field(8, ge=1, le=20)
    use_cache: bool = True
    conversation_id: UUID | None = None
    debug: bool = False                        # adds retrieval trace, admin
role only

class Citation(Strict):
    citation_id: UUID
    doc_id: UUID
    doc_title: str
    doc_version: int
    chunk_id: UUID
    page_no: int
    bboxes: list[BBox]                         # highlight regions in the PDF
viewer
    quote: str                                 # verbatim span from the
source
    quote_start: int; quote_end: int           # char offsets inside the
chunk text
    entailment_score: float = Field(ge=0, le=1)

class ClaimVerdict(Strict):
    claim_id: UUID
    ordinal: int
```

```
    text: str
    verdict: Literal["supported", "partially_supported", "unsupported",
"contradicted"]
    confidence: float = Field(ge=0, le=1)
    citation_ids: list[UUID]
    note: str | None = None                    # shown on hover, e.g. "omits
the exception in 28(4)"

class QueryResponse(Strict):
    query_id: UUID
    answer_id: UUID
    status: Literal["verified", "partial", "abstained"]
    answer: str                                # unsupported claims removed
or flagged inline
    claims: list[ClaimVerdict]
    citations: list[Citation]
    hallucination_score: float = Field(ge=0, le=1)   # 0 best
    abstain_reason: str | None = None
    cache: Literal["hit_l1", "hit_semantic", "miss"]
    corpus_version: int
    timings_ms: dict[str, float]               # embed, cache, dense, sparse,
rrf, rerank, synth, verify
    model_sig: dict[str, str]
    retrieval_trace: list[dict] | None = None  # only when debug=true
```

SSE events (`mode=fast`, streaming): `meta` (ids, cache status), `retrieval` (citation previews as soon as reranking ends), `token` (answer text delta, marked `pending`), `claim_verdict` (one per claim as the judge finishes), `final` (the full `QueryResponse`), `error`. In `strict` mode only `meta`, `retrieval`, `final` and `error` are sent. Streams resume via `Last-Event-ID` using the Redis event stream.

4.4 `/api/audit`

```
class AuditQuery(Strict):                      # query parameters for GET
/api/audit
    q: str | None = None
    status: Literal["verified", "partial", "abstained"] | None = None
    min_score: float | None = Field(None, ge=0, le=1)   # hallucination score
at or above
    user_id: str | None = None
    since: datetime | None = None; until: datetime | None = None
    limit: int = Field(50, ge=1, le=200); cursor: str | None = None
```

```
class AuditRecord(Strict):
    answer_id: UUID; query_id: UUID; question: str; status: str
    hallucination_score: float; user_id: str; created_at: datetime
    cache: str; corpus_version: int

class AuditPage(Strict):
    items: list[AuditRecord]; next_cursor: str | None

class DebateTurn(Strict):
    agent: Literal["claim_extractor", "advocate", "skeptic", "judge"]
    claim_id: UUID | None
    content: dict                               # structured output of that
agent

class AuditReplay(Strict):                      # GET /api/audit/{answer_id}
    response: QueryResponse
    evidence: list[dict]                        # every reranked chunk shown
to the synthesizer
    debate: list[DebateTurn]
    retrieval_trace: list[dict]
    prompts: dict[str, str]                     # prompt template ids +
versions
    chain_seq: int; row_hash: str

class AuditVerifyRequest(Strict):               # POST /api/audit/verify
    answer_id: UUID | None = None               # re-run on stored answer,
current corpus
    text: str | None = Field(None, max_length=20000)   # or verify an
external draft
    filters: Filters = Field(default_factory=Filters)

    @model_validator(mode="after")
    def one_of(self):
        if (self.answer_id is None) == (self.text is None):
            raise ValueError("provide exactly one of answer_id or text")
        return self

class IntegrityReport(Strict):                  # GET /api/audit/integrity
    ok: bool; rows_checked: int; first_bad_seq: int | None = None
```

`POST /api/audit/verify` with `text` lets a reviewer paste a human-written or third-party draft and get claim-level verdicts and citations against the corpus. It returns an `AuditReplay`.

4.5 `/api/graph`

```
class GraphParams(Strict):                      # query parameters for GET
/api/graph
    scope: Literal["documents", "chunks", "clusters"] = "documents"
    collection_id: str | None = None
    doc_id: UUID | None = None                  # neighborhood of one document
    min_weight: float = Field(0.75, ge=0, le=1)   # cosine threshold for
'similar' edges
    limit: int = Field(2000, ge=1, le=10000)

class GraphNode(Strict):
    id: UUID; kind: Literal["document", "chunk", "cluster"]
    label: str; x: float; y: float              # precomputed UMAP layout
    cluster_id: int | None = None; size: float = 1
    meta: dict = Field(default_factory=dict)   # doc_id, page, classification

class GraphEdge(Strict):
    src: UUID; dst: UUID
    kind: Literal["similar", "cites", "supersedes", "references"]
    weight: float = Field(ge=0, le=1)

class GraphResponse(Strict):
    graph_version: int; nodes: list[GraphNode]; edges: list[GraphEdge]
    truncated: bool = False
```

4.6 Error handling

Errors use RFC 9457 `application/problem+json`. One exception handler maps domain exceptions to the model below; unhandled exceptions return `500 INTERNAL` with only the `request_id`, never a stack trace.

```
class FieldError(Strict):
    loc: list[str | int]; msg: str; type: str

class ErrorBody(Strict):
    type: str                                   # e.g.
https://docmind.local/errors/rate-limited
    title: str; status: int
    code: str                                   # stable machine code, table
below
    detail: str
    request_id: str
    retry_after_s: int | None = None
    errors: list[FieldError] | None = None
```

code	HTTP	Raised when	Client behaviour
`INVALID_REQUEST`	422	Pydantic validation fails	Fix fields listed in `errors`
`UNSUPPORTED_MEDIA_TYPE`	415	MIME or magic bytes not allowed	Convert the file
`FILE_TOO_LARGE`	413	Over 200 MB	Split the file
`DUPLICATE_DOCUMENT`	409	Same sha256 already indexed for the tenant	Use `duplicate_of`
`UNAUTHENTICATED` / `FORBIDDEN`	401 / 403	Bad token, or ACL excludes the resource	Re-authenticate; request access
`NOT_FOUND`	404	Unknown id, or id outside the tenant (never reveals existence)	n/a
`RATE_LIMITED`	429	Redis counter exceeded	Wait `Retry-After` seconds
`INDEX_UNAVAILABLE`	503	Qdrant unreachable or circuit open	Retry with backoff

code	HTTP	Raised when	Client behaviour
`LLM_UNAVAILABLE`	503	vLLM queue full or down	Retry; fast retrieval-only fallback is offered in the UI
`VERIFIER_TIMEOUT`	200	Debate exceeded budget (default 8 s)	Not an error: response `status=partial`, unverified claims flagged
`PARSE_FAILED`, `DOCUMENT_ENCRYPTED`	in job `error`	Corrupt or password-protected PDF	Re-upload; unlock the file
`INTERNAL`	500	Unhandled	Quote `request_id` to support

An empty or irrelevant retrieval set is **not** an error: it returns `200` with `status=abstained` and an `abstain_reason`, because abstention is a valid, auditable answer.

5. Core Algorithms & Logic

5.1 Reciprocal Rank Fusion (RRF)

Dense and BM25 scores live on incompatible scales (cosine similarity versus unbounded term-weight sums), so DocMind fuses **ranks**, not scores. Each document earns a weighted reciprocal-rank credit from every list it appears in.

```
\mathrm{RRF}(d) = \sum_{r \in R} \frac{w_r}{k + \mathrm{rank}_r(d)}, \qquad k = 60
```

`k` damps the advantage of the very top ranks; 60 is the standard default and is re-tuned over {20, 60, 100} on the gold set. A document missing from a list contributes nothing for that list.

Worked example. Document A is rank 1 dense and rank 5 sparse: 1/61 + 1/65 = 0.03178. Document B is rank 2 dense only: 1/62 = 0.01613. Document C is rank 1 sparse only: 1/61 = 0.01639. Final order is A, C, B. Agreement across both retrievers beats a single high rank, and C edges out B because rank 1 beats rank 2.

```
import asyncio, re
from collections import defaultdict
from qdrant_client import AsyncQdrantClient, models

IDENT = re.compile(r"""(\b[A-Z]{2,}[- ]?\d[\w.\-]*\b|\b(?:Art(?:icle)?
|Sec(?:tion)?|Clause)\.?\s*\d[\w().]*|\b\d+(?:\.\d+){1,}\b|"[^"]+")""")

def choose_weights(question: str) -> dict[str, float]:
    """Exact identifiers favour BM25; long natural-language questions favour
dense."""
    if IDENT.search(question):
        return {"dense": 1.0, "sparse": 1.5}
    if len(question.split()) > 12:
        return {"dense": 1.0, "sparse": 0.7}
    return {"dense": 1.0, "sparse": 1.0}

def rrf_fuse(rankings: dict[str, list[str]], weights: dict[str, float],
             k: int = 60, top_n: int = 40) -> list[dict]:
    scores: dict[str, float] = defaultdict(float)
    ranks: dict[str, dict[str, int]] = defaultdict(dict)
    for name, ids in rankings.items():
        w = weights.get(name, 1.0)
        for rank, pid in enumerate(ids, start=1):     # rank is 1-based
            scores[pid] += w / (k + rank)
            ranks[pid][name] = rank
    # Deterministic: score desc, then best single rank, then id
    order = sorted(scores, key=lambda p: (-scores[p], min(ranks[p].values()),
p))
    return [{"id": p, "rrf": scores[p], "ranks": ranks[p]} for p in
order[:top_n]]

async def hybrid_retrieve(qdr: AsyncQdrantClient, col: str, dense_vec,
sparse_vec,
                          flt: models.Filter, question: str) -> list[dict]:
    common = dict(collection_name=col, query_filter=flt, limit=50,
with_payload=False)
    dense_req = qdr.query_points(
        query=dense_vec, using="dense",
        search_params=models.SearchParams(
            hnsw_ef=128,
            quantization=models.QuantizationSearchParams(rescore=True,
oversampling=2.0)),
        **common)
```

```
    sparse_req = qdr.query_points(query=sparse_vec, using="bm25", **common)
    dense, sparse = await asyncio.gather(dense_req, sparse_req)   # parallel,
~max(a,b) not a+b
    fused = rrf_fuse(
        {"dense": [str(p.id) for p in dense.points],
         "sparse": [str(p.id) for p in sparse.points]},
        choose_weights(question))
    return fused          # payloads for the 40 ids are fetched in ONE
batched retrieve() call
```

Pipeline after fusion

1. Fetch payloads for the fused top 40 in one `retrieve` call.
2. Cross-encoder (bge-reranker-v2-m3) scores `(question, chunk text)` pairs in a single fp16 batch; keep the top 8.
3. **Evidence gate.** If the best reranker score is below a calibrated floor (initial value 0.30 on the sigmoid scale, tuned on the gold set), skip the LLM and return `abstained` with reason `no_relevant_evidence`. This is the cheapest hallucination defence.
4. Replace each surviving child with its parent chunk (about 1024 tokens), dedupe overlapping parents, drop superseded versions unless requested.
5. Order evidence by document and page, not by score, so the synthesizer reads clauses in their natural sequence. Label blocks `[E1]` to `[E8]`.

Server-side fusion (`prefetch` + `FusionQuery(fusion=Fusion.RRF)`) is kept as a feature-flagged fallback with identical `k` ; it saves one hop but cannot apply per-query weights.

5.2 Dual-agent debate for citation verification

The synthesizer is **not** trusted. After it writes an answer, a verification pipeline attacks every claim. Cheap deterministic checks run first, then two adversarial agents, then a judge. The judge never sees fluent reasoning alone: it sees verbatim spans and a system flag for any quote that does not exist in the evidence.

Stage	Actor	Type	Output
0	Synthesizer	LLM, T=0.1	Sentences, each with `[E#]` tags and a verbatim `quote`
1	Claim extractor	LLM, T=0	Atomic, self-contained claims with flags `has_number` , `is_obligation`

Stage	Actor	Type	Output
2	Deterministic checks	Code	`invalid_quote` (quote not found after whitespace normalization), `number_mismatch`, `identifier_mismatch`
3a	Advocate	LLM, T=0.2	Strongest verbatim support per claim, searching all evidence
3b	Skeptic	LLM, T=0.2	Contradictions, omitted qualifiers, scope or version mismatch, overgeneralization
3c	NLI check (optional)	DeBERTa-v3-class model	Entailment probability of claim given quoted span; an independent, non-LLM vote
4	Judge	LLM, T=0	Verdict, confidence, citations, one-line rationale, suggested fix
5	Gate and repair	Code	Pass, repair once, or abstain

Agents 3a and 3b run concurrently, one request per claim, with vLLM prefix caching so the shared evidence block is prefilled once. All agent outputs use vLLM guided JSON decoding so parsing cannot fail.

Prompts (templates are versioned; the version id is stored in `model_sig`):

```
[SYNTHESIZER]
You are a compliance research assistant. Answer ONLY from the EVIDENCE
blocks.
1. End every sentence with its evidence tags, e.g. [E3] or [E2][E5].
2. For every sentence give a verbatim quote (max 40 words) copied exactly
from a tagged block.
3. Never use outside knowledge. Never state a number, date, section number,
or obligation
   that is not written in the evidence.
4. Keep every qualifier: except, unless, subject to, thresholds, effective
dates.
5. If blocks conflict, state the conflict and cite both.
6. If the evidence does not answer the question, return status
'insufficient_evidence'
   and say precisely what is missing. Do not guess.
```

```
Output JSON: {status, sentences:[{text, evidence_ids:[...], quote}]}

[CLAIM EXTRACTOR]
Split ANSWER into atomic claims: one checkable assertion each. Make every
claim
self-contained: resolve pronouns and include the subject, jurisdiction, date,
and number
it refers to. Keep the original [E#] tags. Do not add, drop, or soften
meaning.
Output JSON: {claims:[{id, text, evidence_ids, has_number, is_obligation}]}

[ADVOCATE]
ROLE: Advocate. For the CLAIM, find the strongest support anywhere in
EVIDENCE, not only
in the tagged blocks. Copy supporting spans VERBATIM. Say whether the span
fully entails
the claim or only partly, and what is missing. A loose paraphrase is a loss;
only exact,
entailing text counts. If there is no support, say found=false.
Output JSON: {claim_id, found, support:[{evidence_id, quote,
covers:'full'|'partial', missing}]}

[SKEPTIC]
ROLE: Skeptic. Assume the CLAIM is wrong until the text proves otherwise.
Search EVIDENCE for:
(a) contradiction; (b) an omitted exception, condition, or threshold; (c)
scope mismatch of
entity, jurisdiction, document version, or effective date; (d) a number,
date, or identifier
that differs; (e) overgeneralization ('may' became 'must', 'some' became
'all').
Quote each offending span VERBATIM. If you find nothing, return no_issue. Do
not invent objections.
Output JSON: {claim_id, verdict_hint:'no_issue'|'issue', issues:[{type,
evidence_id, quote, explanation}]}

[JUDGE]
ROLE: Judge. Inputs: CLAIM, ADVOCATE output, SKEPTIC output, SYSTEM FLAGS,
and the full text of
the quoted evidence blocks. Decide using ONLY that text.
- supported: quoted spans entail every part of the claim; skeptic issues are
absent or refuted by the text.
- partially_supported: part is entailed; a part or a qualifier is not stated.
```

```
- unsupported: no span entails the claim.
- contradicted: the evidence states the opposite.
Any quote flagged invalid_quote is void. Do not reward fluent argument;
reward exact text.
Output JSON: {claim_id, verdict, confidence, citations:[{evidence_id,
quote}], rationale (max 40 words), fix}
```

```
async def verify_answer(answer, evidence, budget_s: float = 8.0):
    claims = await extract_claims(answer)                          # stage 1
    flags = {c.id: deterministic_checks(c, evidence) for c in claims}   #
stage 2
    async with asyncio.timeout(budget_s):
        adv, skp = await asyncio.gather(run_advocate(claims, evidence),
                                        run_skeptic(claims, evidence))  #
stage 3
        nli = await run_nli(claims, adv) if settings.nli_enabled else {}
        verdicts = await run_judge(claims, adv, skp, flags, nli, evidence)  #
stage 4
    return score_and_gate(verdicts)                                # stage 5
```

On `TimeoutError` the pipeline returns the claims judged so far and marks the rest `unverified`, which yields `status=partial`.

5.3 Hallucination score and gate

Each claim gets a risk `r` from its verdict: 0 for supported, 0.5 for partially supported, 1 for unsupported or contradicted. A judge confidence below 0.6 raises `r` to at least 0.5. Claims carrying a number, date, identifier or obligation (`must`, `shall`, `prohibited`) weigh double, because errors there carry compliance cost.

```
H = \frac{\sum_i w_i \, r_i}{\sum_i w_i}, \qquad w_i = \begin{cases} 2 &
\text{numeric or obligation claim} \\ 1 & \text{otherwise} \end{cases}
```

Any `contradicted` claim forces `H = max(H, 0.60)`.

Outcome	Condition	Behaviour
`verified`	H ≤ 0.10 and no unsupported or contradicted claim	Show answer, write to semantic cache

Outcome	Condition	Behaviour
Repair (once)	Some claims failed but the evidence contains support or a fix	Re-synthesize with the judge's `fix` notes, forbidden to add new claims; re-verify only changed claims
`partial`	After removal or flagging, remaining H ⩽ 0.40	Show verified claims; failed claims struck through with the skeptic's note; not cached
`abstained`	Evidence gate fails, zero supported claims, or pruned H > 0.40	Show what was searched and what is missing; not cached

The thresholds 0.10 and 0.40 are starting values. They are calibrated against the adversarial gold set (Section 8) so that the unsupported-claim leak rate stays at or below 0.5%.

5.4 Known limits of the debate design

- Agents share one base model, so correlated blind spots remain. The deterministic checks and the optional NLI model exist to break that correlation.
- Verification can only test claims against the retrieved evidence. A relevant clause that retrieval missed cannot be caught here; recall@20 is measured separately.
- OCR errors propagate. Spans from low-confidence OCR pages (`ocr_conf < 0.80`) are shown with a warning and the judge's confidence is capped at 0.7.